

Lecture 6

Verilog HDL

Peter Cheung
Imperial College London

URL: www.ee.imperial.ac.uk/pcheung/teaching/EE2_CAS/
E-mail: p.cheung@imperial.ac.uk

This lecture is about Verilog HDL, which, together with another language VHDL, are the most popular hardware languages used in industry.

Verilog is only a tool; this course is about digital electronics. Therefore, I will NOT be going through Verilog as in a programming course - it would have been extremely boring for both you and me if I did. Instead, you will learn about Verilog through examples. I will then point out various language features along the way. What it means is that the treatment of Verilog is NOT going to be systematic or comprehensive – there will be lots of features you won't know about Verilog. However, you will learn enough to specify and design reasonably sophisticated digital circuits, and you should gain enough confidence to learn the rest by yourself.

There are many useful online resources available on details of Verilog syntax etc.. Look it up as you need to and you will learn how to design digital circuit using Verilog through designing real circuits.

Lecture Objectives

- ◆ By the end of this lecture, you should understand:
 - The basic structure of a module specified in Verilog HDL
 - Commonly used syntax of Verilog HDL
 - Continuous vs Procedural Assignments
 - **always** block in Verilog and sensitivity list
 - Nets vs variables
 - The use of arithmetic and logic operations in Verilog
 - The danger of **incomplete specification**
 - How to specify **clocked circuits**
 - Differences between **blocking** and **nonblocking** assignments

Here is a list of lecture objectives. They are provided for you to reflect on what you are supposed to learn, and not as an introduction to this lecture.

I want, by the end of this lecture, to give you some idea about the basic **structure** and **syntax** of Verilog. I want to convince you that schematic capture is NOT a good way to design digital circuits. Finally, I want you to appreciate how to use Verilog to specify a piece of hardware at **different levels of abstraction**.

Schematic vs HDL

Schematic

- ✓ Good for multiple data flow
- ✓ Give overview picture
- ✓ Relate directly to hardware
- ✓ Don't need good programming skills
- ✓ High information density
- ✓ Easy back annotations
- ✓ Useful for mixed analogue/digital
- ✗ Not good for algorithms
- ✗ Not good for datapaths
- ✗ Poor interface to optimiser
- ✗ Poor interface to synthesis software
- ✗ Difficult to reuse
- ✗ Difficult to parameterise

HDL

- ✓ Flexible & parameterisable
- ✓ Excellent input to optimisation & synthesis
- ✓ Direct mapping to algorithms
- ✓ Excellent for datapaths
- ✓ Easy to handle electronically (only needing a text editor)
- ✗ Serial representation
- ✗ May not show overall picture
- ✗ Need good programming skills
- ✗ Divorce from physical hardware

You are very familiar with schematic capture. However modern digital design methods in general DO NOT use schematics. Instead an engineer would specify the design requirement or the algorithm to be implemented in some form of computer language specially designed to describe hardware. These are called “Hardware Description Languages” (HDLs).

The most important advantages of HDL as a means of specifying your digital design are: 1) You can make the design take on parameters (such as number of bits in an adder); 2) it is much easier to use compilation and synthesis tools with a text file than with schematic; 3) it is very difficult to express an algorithm in diagram form, but it is very easy with a computer language; 4) you can use various datapath operators such as +, * etc.; 5) you can easily edit, store and transmit a text file, and much hardware with a schematic diagram.

For digital designs, schematic is NOT an option. Always use HDL. In this lecture, I will demonstrate to you why with an example.

Verilog HDL

- ◆ Similar to C language to describe/specify hardware
- ◆ Description can be at different levels:
 - **Behavioural level**
 - **Register-Transfer Level (RTL)**
 - **Gate Level**
- ◆ Not only a specification language, also with associated **simulation environment**
- ◆ Easier to learn and “lighter weight” than its competition: VHDL
- ◆ Very popular with chip designers

- ◆ For this lecture, we will:
 - ❑ Learn through examples and practical exercises
 - ❑ Use two examples: 2-to-1 multiplexer and 7 segment decoder

I have chosen to use Verilog HDL as the hardware description language for this module. Verilog is very similar to the C language, which you should already know from last year. However, you must always remember that YOU ARE USING IT TO DESCRIBE HARDWARE AND NOT AS A COMPUTER PROGRAMME.

You can use Verilog to describe your digital hardware in three different level of abstraction:

1) Behavioural Level – you only describe how the hardware should behave without ANY reference to digital hardware.

2) Register-Transfer-Level (RTL) – Here the description assumes the existence of registers and these are clocked by a clock signal. Therefore digital data is transferred from one register to the next on successive clock cycles. Timing (in terms of clock cycles) is therefore explicitly defined in the Verilog code. This is the level of design we use most frequently in this course.

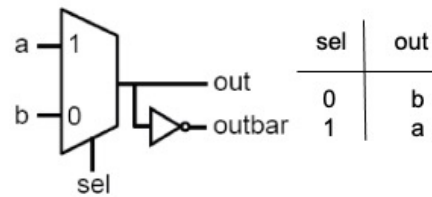
3) Gate Level – this is the lowest level description where each gate and its interconnection are explicitly specified.

Verilog is not only a specification language which tells the CAD system what hardware is supposed to do, it also includes a complete simulation environment. A Verilog compiler does more than mapping your code to hardware, it also can **simulate** (or execute) your design to predict the behaviour of your circuit. It is the predominant language used for chip design.

You will learn Verilog through examples and exercises, not through lecture. However, I will spend just this lecture to cover the basics of Verilog.

Structure of a Module

- Verilog design contains **interconnected modules**
- A module has collections of low-level gates, statements, and other modules
- Here is an example of a simple module that describes a 2-to-1 multiplexer:



```
// Function: 2-to-1 multiplexer
module mux2to1 (out, outbar,
               a, b, sel);
```

```
    output out, outbar;
    input  a, b, sel;
```

```
    assign out = sel ? a : b;
    assign outbar = ~out;
```

```
endmodule
```

- // to end-of-line is comment. Can also use /* ... */ for multiline comments
- Declare and name module; list its ports; terminate with ‘;’

- Specify port as input, output (or inout if bidirectional)

- Express modules behaviour; each statement executes in parallel; **ORDER DOES NOT MATTER**

This is a Verilog module that specifies a 2-to-1 multiplexer. It is rather similar to a C function (except for the **module** keyword).

It is important to remember the basic structure of a Verilog module. There is a module name: **mux2to1**. There is a list of interface ports: 3 inputs **a**, **b** and **sel**, and 2 outputs **out** and **outbar**. Always use meaningful names for both module name and variable names.

You must specify which port is input and which port is output, similar to the data type declaration in a C programme.

Finally, the 2-to-1 multiplexing function is specified in the **assign** statement with a construct that is found in C. This is a **behavioural** description of the multiplexer – no gates are involved.

The last statement specifies the relationship between **out** and **outbar**. It is important to remember that Verilog describes **HARDWARE** not instruction code. The two **assign** statements specify hardware that “execute” or perform the two hardware functions in parallel. Therefore their **order does not matter**.

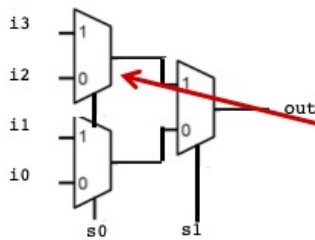
Continuous Assignment

```
module mux2to1 (out, outbar,
               a, b, sel);

    output out, outbar;
    input  a, b, sel;

    assign out = sel ? a : b;
    assign outbar = ~out;

endmodule
```



- ◆ Keyword **assign** specifies **continuous assignment** to describe combinational logic
- ◆ Right-hand expression continuously evaluated responding to input changes immediately
- ◆ Left-hand is a net driven with evaluated value
- ◆ Left side must be a scalar, a net or a concatenation of nets and vector nets. (nets, vectors etc. will be described later)
- ◆ All continuous assignments execute in parallel
- ◆ Operators in expressions are low-level:
 - Conditional assignment: (cond)? vTrue: vFalse
 - Boolean: ~, &, |,
 - Arithmetic: +, -, *
- ◆ Operators can be nested. For example, here is a 4-to-1 mux:

```
assign out = s1 ? (s0 ? i3 : i2) : (s0 ? i1 : i0);
```

Continuous assignment specifies combinational circuits – output is continuously reflecting the operations applied to the input, just like hardware.

Remember that unlike a programming language, the two continuous assignment statements here ARE specifying hardware in PARALLEL, not in series.

Here we also see the conditional statement that is found in C. This maps perfectly to the function of a 2-to-1 multiplexer in hardware and is widely used in Verilog.

Furthermore, there are many other Boolean and arithmetic operators defined in Verilog (as in C). Here is a quick summary of all the Verilog operators (used in an expression).

(What is “reduction and &”? I want you to find this yourself online.)

{}, {}	concatenation
+ - * /	arithmetic
%	modulus
> >= < <=	relational
!	logical negation
&&	logical and
	logical or
==	logical equality
!=	logical inequality
===	case equality
!==	case inequality
~	bit-wise negation
&	bit-wise and
	bit-wise inclusive or
^	bit-wise exclusive or
^~ or ^~	bit-wise equivalence

&	reduction and
~&	reduction nand
	reduction or
~	reduction nor
^	reduction xor
~^ or ^~	reduction xnor
<<	left shift
>>	right shift
? :	condition
or	event or

Description at Gate Level

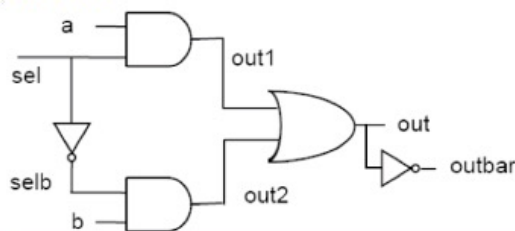
```
// Function: 2-to-1 multiplexer as gates
```

```
module mux_gate (out, outbar,  
                 a, b, sel);
```

```
    output out, outbar;  
    input  a, b, sel;  
    wire   out1, out2, selb;
```

```
    not i1 (selb, sel);  
    and a1 (out1, a, sel);  
    and a2 (out2, b, selb);  
    or o1 (out, out1, out2);  
    not i2 (outbar, out);
```

```
endmodule
```



- ◆ Built-in logic gate primitives:
□ and, nand, or, nor, xor, xnor, not, buf
- ◆ Can use arbitrary number of inputs, e.g. and gate_name (out, in1, in2, in3, ...)
- ◆ Tri-state buffers: bufif1 and bufif0
- ◆ Connect gates with nets using declaration keyword wire
- ◆ **and a1 (out1, a, sel);**
- ◆ This statement is an **instantiation** of an AND gate. **a1** is the name of THIS particular AND gate.

PYKC 1 Nov 2021

EE2 – Circuits & Systems

Lecture 6 Slide 7

While the previous Verilog code for the 2-to-1 mux only specifies “**behaviour**”, here is one that specifies a gate implementation of the same circuit. Three types of gates are used: **and**, **or** and **not** gates. There are internal nets (declared as **wire**) which must also be declared and are used to connect gates together.

Keywords such as **and**, **or** and **xor** are special – they specify actual logic gates. They are also special in that the number of inputs to the and-gate can be 2, 3, 4, Any length!

Note that this module uses TWO AND gates, and they have different names: a1 and a2. These are TWO **separate instances** of the AND gate. In software, “calling” a function simply execute the same piece of programme code. Here the two statements “**and a1 (out1, ...)**” and “**and a2 (out2 ...)**” produce two separate piece of hardware. We say that each line is “**instantiating**” an AND gate.

Wiring up the gates is through the use of ports and wires, and depends on the positions of these “nets”. For example, **out1** is the output net of the AND gate **a1**, and it is connected to the input of the **OR** gate **it** by virtual of its location in the gate port list.

Procedural Assignment using “always”

- ◆ Keyword **always** and **initial** are used to define **procedural assignment**, similar to procedures in software
- ◆ Good for behavioural description of hardware including combinational and sequential logic
- ◆ Support richer C-like constructs such as **if**, **for**, **while**, **case** etc.

<pre> module mux_2_to_1 (out, outbar, a, b, sel); output out, outbar; input a, b, sel; reg out, outbar; always @ (a or b or sel) begin if (sel) out = a; else out = b; outbar = ~out; end endmodule </pre>	◆ Declarations same as before ◆ Assignment inside an always block must be declared as variable data type such as reg (see later) ◆ always block runs once whenever a signal in the sensitivity list changes value ◆ Statements inside the always block are executed sequentially. ORDER MATTERS! ◆ Sandwiched between begin/end
---	--

PYKC 1 Nov 2021

EE2 – Circuits & Systems

Lecture 6 Slide 8

So far we have used Verilog in very hardware specific way. “assign” and using gate specification are special to Verilog. You do not find these in C.

Here is something that is more like C – and it is called “**procedural assignment**”. Typically we use something called “**always**” block to specify a “**procedure**”, i.e. a collection of sequential statements which are sandwiched between the **begin-end** construct.

The **always** block needs a **sensitivity list** – a list of signals which, if ANY of these signals changes, the **always** block will be invoked. You may read this block as:

“always at any changes in nets **a**, **b** or **sel**, do the bits between **begin** and **end**”.

Actually, if you are defining a combinational circuit module, an even better way to define the always block is to use:

..... always @ (*) // always at any change with any input signal

Inside the **begin-end** block, you are allowed to use C-like statements. In this case, we use the **if-else** statement. All statements inside the **begin-end** block are **executed sequentially**.

Verilog “register” is NOT what it appears!

- ◆ Registers normally represents storage elements in digital logic, they need clock signals to update their output value
- ◆ In Verilog **reg** is NOT the same as a digital register, it is used only to declare a variable that holds a value
- ◆ Values of variables (declared as **reg**) can be changed anytime in a simulation, and can be used for nets of a combinational circuit
- ◆ In other words, in Verilog, **reg** is similar to declarations such as **int**, **real** etc. When you use:

```
reg signal_a;
```

whether a physical register (or D-FF) is synthesized to store **signal_a** or not would depend on the rest of the Verilog code.

Note that Verilog keyword **reg** does not implies that there is a register created in the hardware. It is much more like declaring a variable that holds a value. It is a rule in Verilog that if you perform an **assignment** to a variable **INSIDE** an **always** block, that variable **MUST be declared a reg**, and NOT a net (**wire**). This is one of the few peculiarities of Verilog that can be confusing to students.

case statement – better alternative to if-else

- ◆ **case** statement can often replace **if-else** construct within an **always** block, and provides better abstraction
- ◆ Here is an example using the mux_2_to_1 module:

```
always @ (a or b or sel)
begin
```

```
    case (sel)
        1'b0: out = b;
        1'b1: out = a;
    endcase
```

```
end
```

```
if (sel) out = a;
else out = b;
```



Notation for numbers:

<size> ' <base> <number>

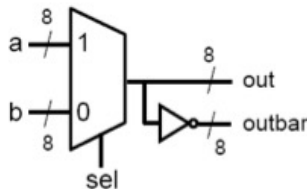
2'b10	2 bit binary, v=2
'b10	Unsize binary 32-bit, v=2
31	Unsize decimal, v=31
8'hAf	8-bit hex, v=175
-16'd47	16-bit negative decimal, v=-47

This is yet another way to specify the MUX circuit. It is still a procedural assignment with the **always** block. However, we replace the **if-else** statement with a “**case**” statement. The case variable is **sel**. Since **sel** is a 1-bit signal (or net), it can only take on 0 or 1.

Note that the various case values can be expressed in different number formats as shown in the slide. For example, consider **2'b10**. The 2 is the number of bits in this number. **'b** means it is specified in binary format. The value of this number is **10** in binary.

n-bit signals - buses

- ◆ Verilog is powerful in specifying multi-bit signals and buses.
- ◆ Here is an example for 8-bit wide 2-to-1 multiplexer:



```
module mux_2_to_1(a, b, out,
                  outbar, sel);
    input [7:0] a, b;
    input sel;
    output [7:0] out, outbar;
    reg [7:0] out;
    always @ (a or b or sel)
    begin
        if (sel) out = a;
        else out = b;
    end
    assign outbar = ~out;
endmodule
```

Concatenate signals using the { } operator

```
assign {b[7:0],b[15:8]} = {a[15:8],a[7:0]};
effects a byte swap
```

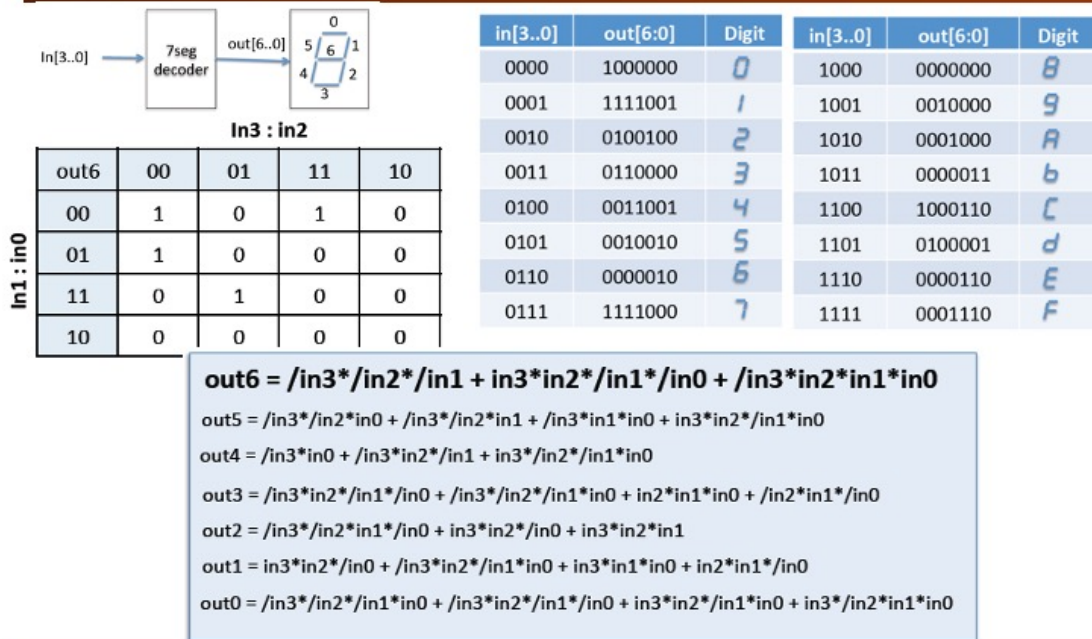
This slide demonstrates why language specification of hardware is so much better than schematic diagram. By simply declaring the signals as a multi-bit bus (8 bits [7:0]), we change this module to one that specifies 8 separate 2-to-1 multiplexers.

Another useful way to specify a bus is using the concatenation operator: { } as shown above.

The concatenation operator is particularly useful in converting digital signals from one word length (i.e. number of bits in a word) to another. For example, to convert an 8-bit unsigned number a[7:0] to a 13-bit unsigned number b[12:0], you can simple do this:

```
assign    b[12:0] = {5'b0, a[7:0]};
```

Putting everything together – 7 seg decoder



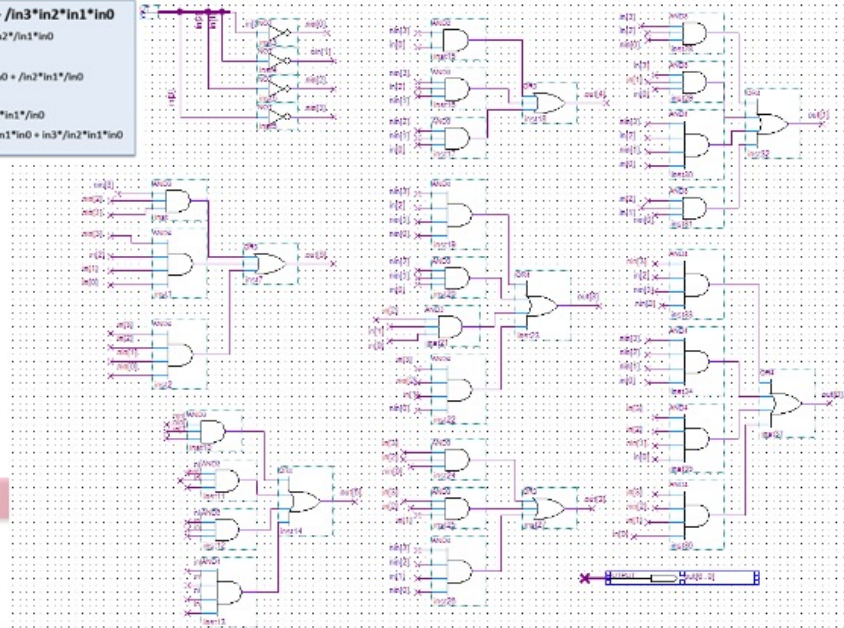
Here is a simple example: the design of a 4-bit hex code to 7 segment decoder. You can express the function of this 7-segment decoder in three forms: 1) as a truth table (note that the segments are low active); 2) as 7 separate K-maps (shown here is for **out[6]** segment only); 3) as Boolean equations.

This is probably the last time you see K-maps. In practical digital design, you would rely heavily on CAD tools. In which case, logic simplifications are done for you automatically – you never need to use K-maps to do Boolean simplification manually!

Method 1: Schematic Entry Implementation

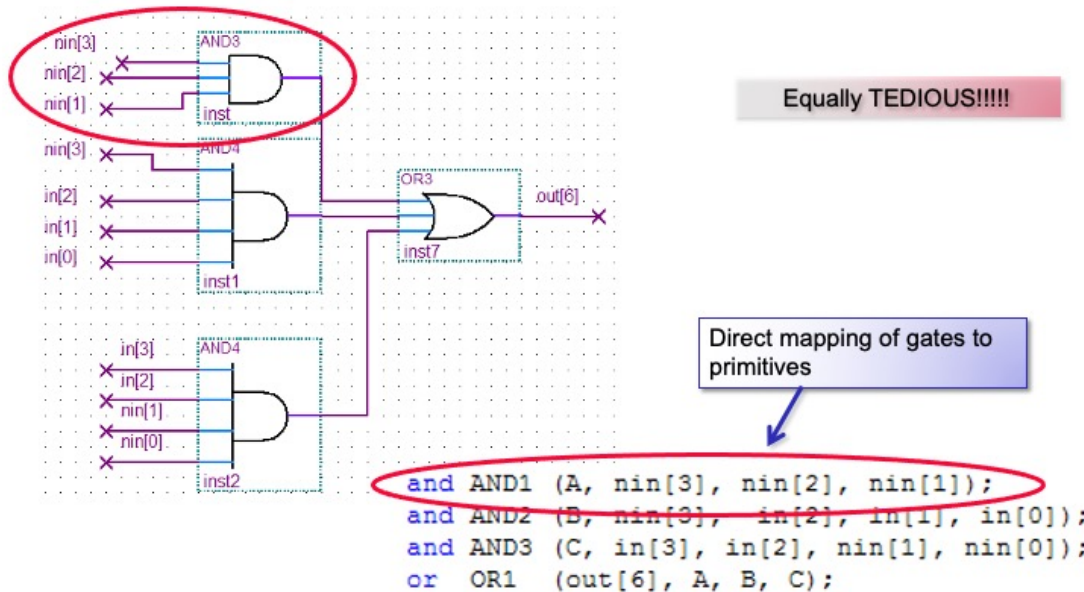
$out6 = /in3*/in2*/in1 + in3*in2*/in1*/in0 + /in3*in2*in1*in0$
 $out5 = /in3*/in2*in0 + /in3*/in2*in1 + /in3*in1*in0 + in3*in2*/in1*in0$
 $out4 = /in3*in0 + /in3*in2*/in1 + in3*/in2*/in1*in0$
 $out3 = /in3*in2*/in1*in0 + /in3*in2*/in1*in0 + in2*in1*in0 + /in2*in1*/in0$
 $out2 = /in3*/in2*in1*in0 + in3*in2*/in0 + in3*in2*in1$
 $out1 = in3*in2*/in0 + /in3*in2*/in1*in0 + in3*in1*in0 + in2*in1*/in0$
 $out0 = /in3*/in2*/in1*in0 + /in3*in2*/in1*in0 + in3*in2*/in1*in0 + in3*/in2*in1*in0$

TEDIOUS!!!!



Here is a tedious implementation in the form of schematic diagram of the 7 segment decoder as interconnected gates. Very hard to do and very prone to errors.

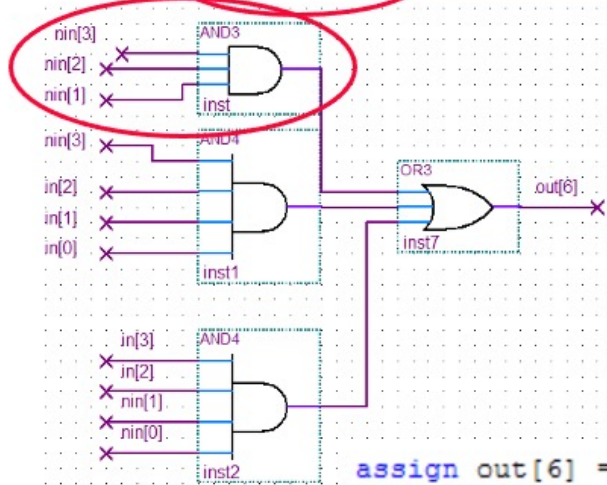
Method 2: Use primitive gates in Verilog



One could take a group of gates and specify the gates in Verilog gate primitives such as **and**, **or** etc. Still very tedious. Here is the implementation for the **out[6]** output.

Method 3: Use continuous assignment in Verilog

$out6 = \sim in3 * \sim in2 * \sim in1 + in3 * in2 * \sim in1 * \sim in0 + \sim in3 * in2 * in1 * in0$



Much Better?

Direct mapping of Boolean equation using continuous assignment..

```
assign out[6] = ~in[3]&~in[2]&~in[1] |
in[3]&in[2]&~in[1]&~in[0] |
~in[3]&in[2]&in[1]&in[0];
```

Instead of specifying each gate separately, here is using continuous assignment statement, mapping the **Boolean equation** direction to a single Verilog statement. This is better.

module & endmodule
sandwich the content of
this hardware module

Hex_to_7seg.v

```

//-----
// Module name: hex_to_7seg
// Function: convert 4-bit hex value to drive 7 segment display
//          output is low active
// Creator:  Peter Cheung
// Version:  1.0
// Date:    22 Oct 2011
//-----
module hex_to_7seg (out,in);

    output [6:0] out;    // low-active output to drive 7 segment display
    input  [3:0] in;     // 4-bit binary input of a hexadecimal number

    assign out[6] = ~in[3]&~in[2]&~in[1] | in[3]&in[2]&~in[1]&~in[0] |
                   ~in[3]&in[2]&in[1]&in[0];
    assign out[5] = ~in[3]&~in[2]&in[0] | ~in[3]&~in[2]&in[1] |
                   ~in[3]&in[1]&in[0] | in[3]&in[2]&~in[1]&in[0];
    assign out[4] = ~in[3]&in[0] | ~in[3]&in[2]&~in[1] | in[3]&~in[2]&~in[1]&in[0];
    assign out[3] = ~in[3]&in[2]&~in[1]&~in[0] | ~in[3]&~in[2]&~in[1]&in[0] |
                   in[2]&in[1]&in[0] | ~in[2]&in[1]&~in[0];
    assign out[2] = ~in[3]&~in[2]&in[1]&~in[0] | in[3]&in[2]&~in[0] |
                   in[3]&in[2]&in[1];
    assign out[1] = in[3]&in[2]&~in[0] | ~in[3]&in[2]&~in[1]&in[0] |
                   in[3]&in[1]&in[0] | in[2]&in[1]&~in[0];
    assign out[0] = ~in[3]&~in[2]&~in[1]&in[0] | ~in[3]&in[2]&~in[1]&~in[0] |
                   in[3]&in[2]&~in[1]&in[0] | in[3]&~in[2]&in[1]&in[0];

endmodule
  
```

good header helps
documenting your code

specify interface to this
module as viewed from
outside

specify a 7-bit output bus,
out[6] ... out[0]

declaration of
input and output
ports

assign used to specify
combinational circuit

PYKC 1 Nov 2021
EE2 – Circuits & Systems
Lecture 6 Slide 16

Here is the complete specification of the hex_to_7seg module using **continuous assignment** statements. It shows how one should write Verilog code with good comments and clear documentation of input and output ports.

Method 4: Power of behavioural abstraction

```

module hex_to_7seg (out,in);
    output [6:0] out;    // low-active output to
    input  [3:0] in;     // 4-bit binary input o

    reg [6:0] out;       // make out a variable

    always @ (in)
        case (in)
            4'h0: out = 7'b1000000;
            4'h1: out = 7'b1111001;    // -- 0 ---
            4'h2: out = 7'b0100100;    // |       |
            4'h3: out = 7'b0110000;    // 5       1
            4'h4: out = 7'b0011001;    // |       |
            4'h5: out = 7'b0010010;    // -- 6 ---
            4'h6: out = 7'b0000010;    // |       |
            4'h7: out = 7'b1111000;    // 4       2
            4'h8: out = 7'b0000000;    // |       |
            4'h9: out = 7'b0011000;    // -- 3 ---
            4'ha: out = 7'b0001000;
            4'hb: out = 7'b0000011;
            4'hc: out = 7'b1000110;
            4'hd: out = 7'b0100001;
            4'he: out = 7'b0000110;
            4'hf: out = 7'b0001110;
        endcase
endmodule

```

BEAUTIFUL !!!

- Direct mapping of truth table to case statement
- Close to specification, not implementation

in[3..0]	out[6:0]	Digit
0000	1000000	0
0001	1111001	1
0010	0100100	2
0011	0110000	3
0100	0011001	4
0101	0010010	5
0110	0000010	6
0111	1111000	7
1000	0000000	8
1001	0010000	9
1010	0001000	A
1011	0000011	b
1100	1000110	c
1101	0100001	d
1110	0000110	E
1111	0001110	F

PYKC 1 Nov 2021

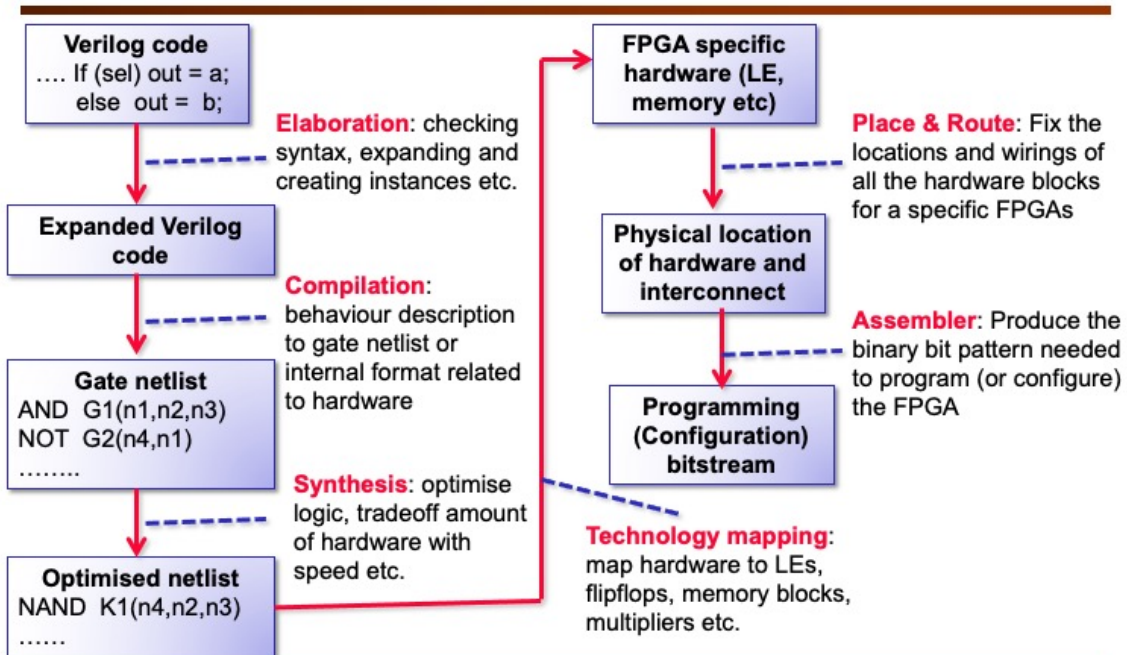
EE2 – Circuits & Systems

Lecture 6 Slide 17

Finally the 4th method is the best. We use the **case** construct to specify the behaviour of the decoder. Here one directly maps the truth table to **the case statement** – easy and elegant.

Instead of using: `always @ (in),` you could also use `always @*`

From Verilog code to FPGA hardware



PYKC 1 Nov 2021

EE2 – Circuits & Systems

Lecture 6 Slide 18

How is a Verilog description of a hardware module turned into FPGA configuration?
This flow diagram shows the various steps taken inside the Quartus Prime CAD system.

Power of Verilog: Integer Arithmetic

- ◆ Arithmetic operations make computation easy:

```
module add32(a, b, sum);  
  input[31:0] a,b;  
  output[31:0] sum;  
  assign sum = a + b;  
endmodule
```

- ◆ Here is a 32-bit adder with carry-in and carry-out:

```
module add32_carry(a, b, cin, sum, cout);  
  input[31:0] a,b;  
  input cin;  
  output[31:0] sum;  
  output cout;  
  assign {cout, sum} = a + b + cin;  
endmodule
```

Verilog is very much like C. However, the declaration of **a**, **b** and **sum** in the **module add32** specifies the data width (i.e. number of bits in each signal **a**, **b** or **sum**). This is often known as a “**vector**” or a “**bus**”. Here the data width is 32-bit, and it is ranging from bit 31 down to bit 0 (e.g. **sum[31:0]**).

You can refer to individual bits using the index value. For example, the least-significant bit (LSB) of **sum** is **sum[0]** and the most-significant bit (MSB) is **sum[31]**. **sum[7:0]** refers to the least-significant byte of **sum**.

The ‘+’ operator can be used for signals of any width. Here a 32-bit add operation is specified. **sum** is also 32-bit in width. However, if **a** and **b** are 32-bit wide, the sum result could be 33-bit (including the carry out). Therefore this operation could result in a wrong answer due to **overflow** into the carry bit. The 33th bit is truncated.

The second example **module add32_carry** shows the same adder but with carry input and carry output. Note the LHS of the **assign** statement. The **{cout, sum}** is a **concatenation** operator – the contents inside the brackets **{ }** are concatenated together, with **cout** is assigned the MSB of the 33th bit of the result, and the remaining bits are formed by **sum[31:0]**.

Different types of Boolean Operators

- ◆ **Bitwise operators:** perform bit-sliced operations on vectors bit by bit
 - $\sim(4'b0101) = \{\sim 0, \sim 1, \sim 0, \sim 1\} = 4'b1010$
 - $4'b0101 \& 4'b0011 = 4'b0001$
- ◆ **Logical operators:** return true or false (1-bit) results
 - $!(4'b0101) = \sim 1 = 1'b0$
- ◆ **Reduction operators:** act on each bit of a SINGLE input vector
 - $\&(4'b0101) = 0 \& 1 \& 0 \& 1 = 1'b0$

Bitwise

$\sim a$	NOT
$a \& b$	AND
$a b$	OR
$a \wedge b$	XOR
$a \sim \wedge b$	XNOR

Logical

$!a$	NOT
$a \&\& b$	AND
$a b$	OR

Note distinction between $\sim a$ and $!a$

Reduction

$\&a$	AND
$\sim \&$	NAND
$ $	OR
$\sim $	NOR
$\wedge$	XOR

There are three different types of Boolean operators:

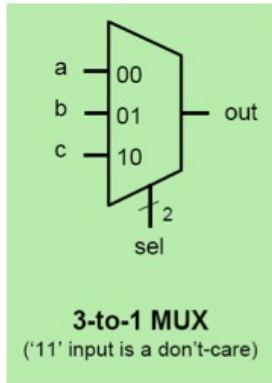
Bitwise operators perform what you would expect as if there are parallel gates used for each bit of the operands. Therefore **$a \& b$** means that each bit from **a** and **b** is passed through an AND-gate.

Logical operators only result in 0 or 1 (i.e. 1-bit result) In this example **$!a$ (not a)** where **$a = 0101$** , will result in first, **a** being evaluated as a logical value (i.e. logical '1' or true). Therefore the result **$\sim a$** is logical 0 (or false).

Reduction operators is applied to a single operand (and sometimes known as unary operators). It performs the operation one-bit at a time to the operand.

Beware of Incomplete Specification

Intention



What you may write, but wrong!

```
module maybe_mux_3to1(a, b, c,
                      sel, out);

    input [1:0] sel;
    input a,b,c;
    output out;
    reg out;

    always @(a or b or c or sel)
    begin
        case (sel)
            2'b00: out = a;
            2'b01: out = b;
            2'b10: out = c;
        endcase
    end
endmodule
```

Assume that we want to specify a 3-to-1 multiplexer as shown on the left. On the right is an attempt to specify this using the always + case construct in Verilog.

The case variable 'sel' is 2-bit wide, and therefore has 4 possibilities. The case statement only specifies three of the four possible cases.

This is known as an **"incomplete specification"**.

In Verilog, there is this rule:

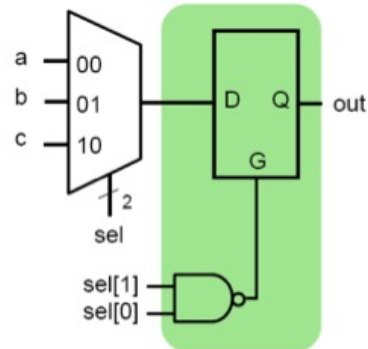
If something is not completely specified, the output must retain its previous value when the unspecified condition occurs.

Incomplete specification: adds unwanted latch circuit

if out is not assigned
during any pass through
the always block, then the
previous value must be
retained!

```
module maybe_mux_3to1(a, b, c,  
                      sel, out);  
    input [1:0] sel;  
    input a,b,c;  
    output out;  
    reg out;  
  
    always @(a or b or c or sel)  
    begin  
        case (sel)  
            2'b00: out = a;  
            2'b01: out = b;  
            2'b10: out = c;  
        endcase  
    end  
endmodule
```

Synthesized Circuit



- ◆ When $sel = 2'b11$, $G = 0$, therefore the latch stores the previous output value as required by Verilog in this situation.

The consequence of this is an unexpected extra latch being added to the hardware. In order to cope with the unspecified condition of $sel = 2'b11$, the output of the MUX is fed to be **latch**.

Noted that a latch is **level-triggered**; a flipflop is **edge-triggered**. A latch has the property that when the gate input **G** is high, **Q = D** (i.e. it is **transparent**: input goes straight to output). If **G** is low, the latch become **opaque**, meaning that it retains the previous value.

The green shaded latch in the diagram and the controlling NAND gate are the unintended consequences of this incompletely specified 3-to-1 multiplexer.

Always avoid incomplete specification

- ◆ Solution 1: Precede all conditionals with a default assignment for all signals:

```
always @(a or b or c or sel)
begin
    out = 1'bx;
    case (sel)
        2'b00: out = a;
        2'b01: out = b;
        2'b10: out = c;
    endcase
end
endmodule
```

- ◆ Solution 2: Fully specify all branches of if-else construct, or include a default statement in case construct:

```
always @(a or b or c or sel)
begin
    case (sel)
        2'b00: out = a;
        2'b01: out = b;
        2'b10: out = c;
        default: out = 1'bx;
    endcase
end
endmodule
```

There are two solutions to avoid the unintended latch being added.

Solution 1 is to put outside the **case** statement a “**default**” value for out. Here **1'bx** (i.e. 'x') means **undefined**.

Solution 2 is better: inside the **case** statement block, always add the **default** line. This will catch ALL the unspecified cases and avoid the introduction of the spurious unintended latches.

Lesson: always include a default assignment in any **case** statement to capture unintended incomplete specification.

How to specify a sequential circuit?

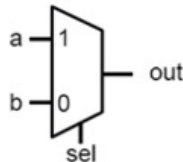
- ◆ Edge-triggered flipflop is specified with: **always @ (posedge clk):**

Combinational cct

```
module combinational(a, b, sel,
                    out);

    input a, b;
    input sel;
    output out;
    reg out;

    always @ (a or b or sel)
    begin
        if (sel) out = a;
        else out = b;
    end
endmodule
```

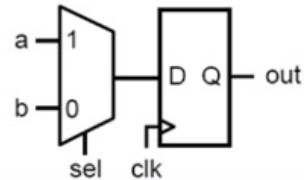


Sequential cct

```
module sequential(a, b, sel,
                 clk, out);

    input a, b;
    input sel, clk;
    output out;
    reg out;

    always @ (posedge clk)
    begin
        if (sel) out <= a;
        else out <= b;
    end
endmodule
```



We have previously seen the 2-to-1 MUX being specified as combinational circuit in Verilog using the **always** construct with the **sensitivity list**.

The right-hand diagram shows how a clocked **sequential circuit** is being specified using **always** block, but with a **sensitivity list** that includes the keyword **posedge** (or **negedge**). Note that the clocking signal **clk** is an arbitrary name – you could call it “fred” or anything else!

The **sensitivity list** NO LONGER contains the input signals **a**, **b** or **sel**. Instead the hardware is specified to be sensitive the positive edge of **clk**. When this happens, the output changes according to the specification inside the **always** block.

Two assignments (“=” and “<=”) are shown here. I will explain the difference between these later.

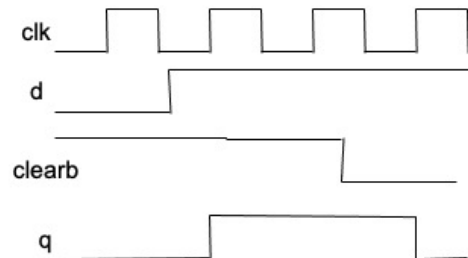
Synchronous clear in D-flipflop

- ◆ **posedge** and **negedge** makes an always block sequential and edge-triggered
- ◆ Sensitivity list in a sequential **always** block determines what circuit is synthesized

D flipflop with **synchronous** clear

```
module dff_sync_clear(d, clearb,
clock, q);
input d, clearb, clock;
output q;
reg q;
always @ (posedge clock)
begin
    if (!clearb) q <= 1'b0;
    else q <= d;
end
endmodule
```

+ve edge on clock triggers action in
always block



- ◆ Beware of **race condition** if you have two or more always blocks – they execute in parallel!

Therefore in Verilog, you specify flipflops using **always block** in conjunction with the keyword **posedge** or **negedge**.

Here is a specification for a D-flipflop with synchronous clear which is low active (i.e. clear the FF when **clearb** is low).

You may have more than one **always** block in a module. But if this is the case, beware that the two **always** blocks will **execute in parallel**. Therefore they must NOT specify the same output, otherwise a **race condition** exists and the result is unpredictable.

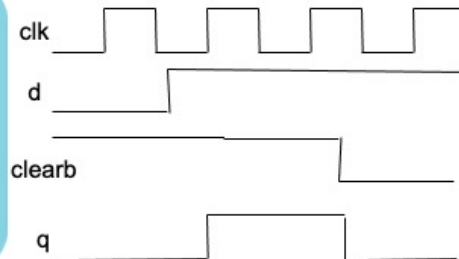
Asynchronous clear in D-flipflop

- ◆ If one signal in the sensitivity list uses **posedge** or **negedge**, ALL signals must also specify an edge. This syntax is wrong: **always @ (clearb or posedge clock)**

D flipflop with **asynchronous** clear

```
module dff_async_clear(d, clearb, clock, q);
input d, clearb, clock;
output q;
reg q;

always @ (negedge clearb or posedge clock)
begin
    if (!clearb) q <= 1'b0;
    else q <= d;
end
endmodule
```



+ve edge on clock OR clearb trigger action in always block

Here is a specification for asynchronous clear of the D-flipflop. Either positive edge on **clock** or negative edge on **clearb** will cause the statements inside the **always** block to take effect.

I must remind everyone that the code shown here is a **specification**. They are **synthesised** into logic circuits – they are NOT executed as in a C programme.

Blocking vs Non-blocking Assignments

- Verilog has two different types of assignments: **blocking** & **nonblocking**.
- Blocking assignments **=** are executed in the order they appear, therefore they are done one after another. Therefore the first statement “blocks” the second until it is done, hence it is called blocking assignments.

<pre>a = b; b = a; // both a & b = b</pre>	blocking
<pre>always @ (a or b or c) begin x = a b; y = a ^ b ^ c; z = b & ~c; end</pre>	blocking
	1. Evaluate $a b$, assign result to x 2. Evaluate $a \wedge b \wedge c$, assign result to y 3. Evaluate $b \wedge (\sim c)$, assign result to z

- Non-blocking assignments **<=** are executed in parallel. Therefore an earlier statement does not block the later statement. Note the subtle effect this has within **always** block:

<pre>a <= b; b <= a; // swap a and b</pre>	Non-blocking
<pre>always @ (a or b or c) begin x <= a b; y <= a ^ b ^ c; z <= b & ~c; end</pre>	Non-blocking
	1. Evaluate $a b$ but defer assignment of x 2. Evaluate $a \wedge b \wedge c$ but defer assignment of y 3. Evaluate $b \wedge (\sim c)$ but defer assignment of z 4. Assign x , y , and z with their new values

PYKC 1 Nov 2021

EE2 – Circuits & Systems

Lecture 6 Slide 27

In Verilog ‘=’ is known as **blocking assignment**. They are **executed in the order** they appear within the Verilog simulation environment. So the first ‘=’ assignment blocks the second one. This is very much like what happens in C codes.

In the top left example, both **a** and **b** eventually have the value **b**.

In the top right example, each statement is evaluated in turn and assignment is performed immediately at the end of the statement.

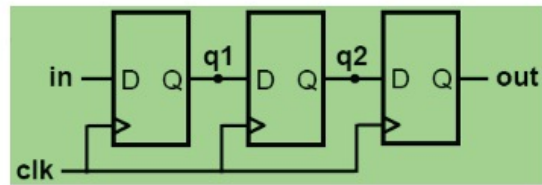
Non-block assignment is ‘<=’, and statements with this assignments are **executed in parallel** (i.e. order do not matter).

In the bottom left example, **a** and **b** are **swapped** over because you can view that the two assignments happen at the same time.

In the bottom right example, three evaluations are made, and the assignment to x , y and z happens at the same time on exiting from the **always** block.

Be careful to use the correct assignment

- ◆ Here are two versions of a 3-stage shift register consisting of 3 flipflops using blocking and nonblocking assignments.
- ◆ Will they give the same results?



```
module blocking(in, clk, out);
    input in, clk;
    output out;
    reg q1, q2, out;
    always @ (posedge clk)
    begin
        q1 = in;
        q2 = q1;
        out = q2;
    end
endmodule
```

```
module nonblocking(in, clk, out);
    input in, clk;
    output out;
    reg q1, q2, out;
    always @ (posedge clk)
    begin
        q1 <= in;
        q2 <= q1;
        out <= q2;
    end
endmodule
```

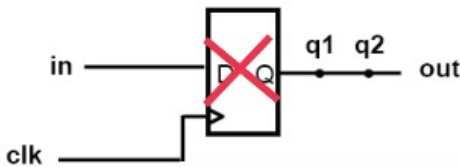
Understanding the difference between '=' and '<=' is important. Suppose we want to specify a three-stage shift register (i.e. three D-FF in series as shown in the schematic).

Here are two possible specification. Which one do you think will create the correct circuit and which one is wrong?

Use NONBLOCKING assignment for sequential logic

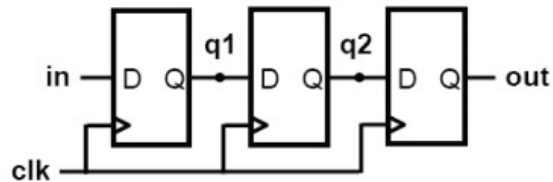
```
always @ (posedge clk)
begin
    q1 = in;
    q2 = q1;
    out = q2;
end
```

- ◆ At each rising clock edge:
q1 = in,
then q2 = q1 = in,
then, out = q2 = q1 = in.
- ◆ Therefore out = in, which is NOT the intention.



```
always @ (posedge clk)
begin
    q1 <= in;
    q2 <= q1;
    out <= q2;
end
```

- ◆ At each rising clock edge, *q1*, *q2* and *out* simultaneously receive the old values of *in*, *q1*, *q2* respectively.



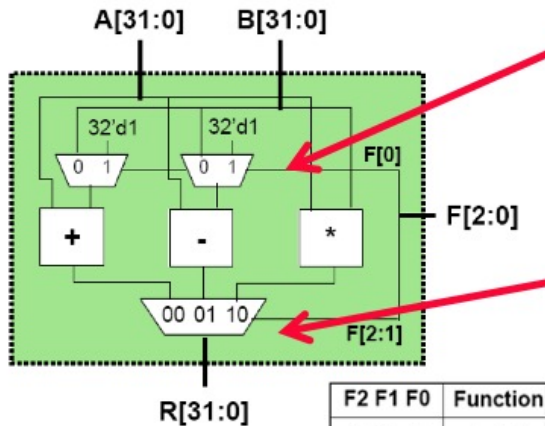
The left hand specification is **wrong**. Since the three assignments are performed in sequence, **out = q2 = q1 = in**. Therefore the resultant circuit is ONE D-flipflop.

The right hand side is **correct**. **q1**, **q2** and **out** are updated simultaneously on exit from the **always** block. Therefore their “original” values MUST be retained. Hence this will result in three D-flipflops being synthesised (i.e. created).

In general, you should always use '**<=**' inside an **always** block to specify your circuit.

A larger example – 32-bit ALU in Verilog

- Here is an 32-bit ALU with 5 simple instructions:



F2	F1	F0	Function
0	0	0	A + B
0	0	1	A + 1
0	1	0	A - B
0	1	1	A - 1
1	0	X	A * B

2-to-1 MUX

```
module mux32two(i0,i1,sel,out);
input [31:0] i0,i1;
input sel;
output [31:0] out;

assign out = sel ? i1 : i0;

endmodule
```

3-to-1 MUX

```
module mux32three(i0,i1,i2,sel,out);
input [31:0] i0,i1,i2;
input [1:0] sel;
output [31:0] out;
reg [31:0] out;

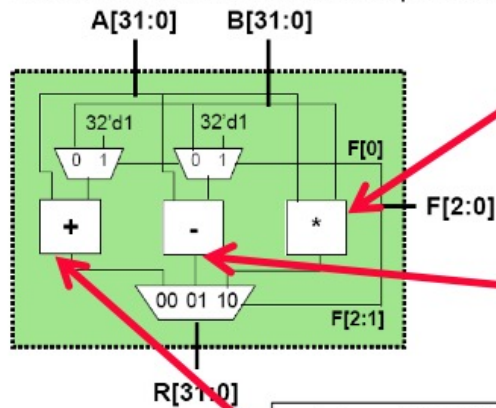
always @ (i0 or i1 or i2 or sel)
begin
case (sel)
2'b00: out = i0;
2'b01: out = i1;
2'b10: out = i2;
default: out = 32'bx;
endcase
end
endmodule
```

Now let us put all you have learned together in specifying (or designing) a 32-bit ALU in Verilog.

There are five operators in this ALU. We assume that there are three arithmetic blocks, and three multiplexers (two 2-to-1 MUX and one 3-to-1 MUX).

The arithmetic modules

- Here is an 32-bit ALU with 5 simple instructions:



```
module mul16(i0,i1,prod);
input [15:0] i0,i1;
output [31:0] prod;

// this is a magnitude multiplier
// signed arithmetic later
assign prod = i0 * i1;

endmodule
```

```
module sub32(i0,i1,diff);
input [31:0] i0,i1;
output [31:0] diff;

assign diff = i0 - i1;

endmodule
```

```
module add32(i0,i1,sum);
input [31:0] i0,i1;
output [31:0] sum;

assign sum = i0 + i1;

endmodule
```

Each hardware block is defined as a Verilog module. So we have the following modules:

mux32two – a 32-bit multiplexer that has TWO inputs

mux32three – a 32-bit multiplexer that has THREE inputs

mul16 – a 16-by-16 binary multiplier that produces a 32-bit product

add32 – a 32-bit binary adder

sub32 – a 32-bit binary subtractor

Top-level module – putting them together

◆ Given submodules:

```
module mux32two(i0,i1,sel,out);
module mux32three(i0,i1,i2,sel,out);
module add32(i0,i1,sum);
module sub32(i0,i1,diff);
module mul16(i0,i1,prod);
```

```
module alu(a, b, f, r);
  input [31:0] a, b;
  input [2:0] f;
  output [31:0] r;
```

```
  wire [31:0] addmux_out, submux_out;
  wire [31:0] add_out, sub_out, mul_out;
```

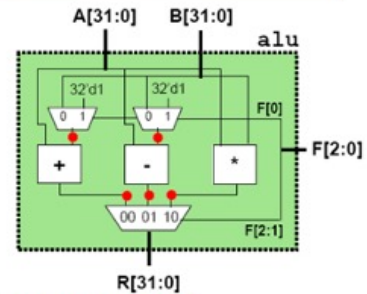
```
  mux32two adder_mux(b, 32'd1, f[0], addmux_out);
  mux32two sub_mux(b, 32'd1, f[0], submux_out);
  add32 our_adder(a, addmux_out, add_out);
  sub32 our_subtractor(a, submux_out, sub_out);
  mul16 our_multiplier(a[15:0], b[15:0], mul_out);
  mux32three output_mux(add_out, sub_out, mul_out, f[2:1], r);
```

endmodule

module names

(unique) instance names

corresponding wires/regs in module alu



Intermediate output nodes

Now let us put all these together.

Note that **mxu32two** is being used twice and therefore this is **instantiated** two times with two different **instance names**: **adder_mux** and **sub_mux**.

Connections between modules are implicit through the use of **signal names**. For example, the 16-bit inputs to the multiplier are taken from the lower 16-bits of **a** and **b** inputs (i.e. **a[15:0]** and **b[15:0]**).